

Docker — Containerizing the Node

Packaging the entire operating environment — interpreter, system libraries, and app — into one portable image, and why a node ships this way.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 15 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Docker · Image · Container · Dockerfile · Layers · Multi-stage build · Base image ·
ENTRYPOINT · Containers vs VMs

Table of Contents

Chapter 15 — Docker: Containerizing the Node	3
15.1 The problem: "works on my machine," one level deeper	3
15.2 Containers, images, and how they differ from VMs	4
15.3 The Dockerfile: a recipe for an image	5
15.4 Reading hathor-core's multi-stage Dockerfile	5
15.5 Why containerize a node, and why Docker?	6
15.6 Bridge — Docker across the project	7
Recap	7

Chapter 15 — Docker: Containerizing the Node

What you'll learn in this chapter

- The problem Docker solves beyond a virtual environment: isolating not just Python packages but the whole operating environment (system libraries, the interpreter, the OS userland).
- The core vocabulary — **image** vs. **container**, **Dockerfile**, **layers**, **base image** — and how a container differs from a virtual machine.
- How `hathor-core`'s **multi-stage** Dockerfile builds a small, clean production image.
- Why a node ships as a container at all, and **why Docker** versus the alternatives.

Chapter 12 isolated Python packages with a virtual environment. But a node needs more than Python packages to run: it needs specific system libraries (the RocksDB C++ library, OpenSSL, graphviz), a specific Python interpreter version, and a predictable operating-system userland around them. A `.venv` doesn't capture any of that. **Docker**¹ does: it packages the entire environment — OS libraries, interpreter, dependencies, and the app — into one portable artifact that runs identically on a laptop, a CI runner, and a production server. This chapter reads `hathor-core`'s real `Dockerfile` to see how. It follows the \$5 primer shape, ending with the "why Docker, not the alternatives" trade-off.

15.1 The problem: "works on my machine," one level deeper

A virtual environment guarantees the same Python packages everywhere (Chapter 12). It does not guarantee the same everything else. The node's RocksDB bindings need the RocksDB C++ system library installed at a specific version; the crypto stack needs OpenSSL; some tooling needs graphviz. These are installed by the operating system's package manager (`apt` on Debian), not by pip — and they differ between a developer's Ubuntu laptop, a teammate's Mac, and a production server. Even the Python interpreter's patch version can differ.

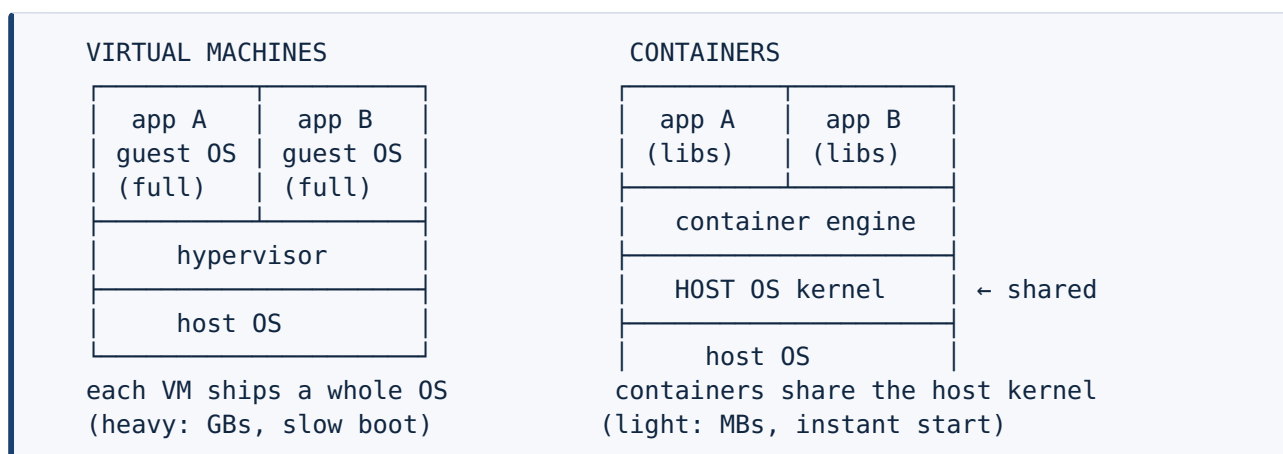
So "works on my machine" can fail one level below the virtual environment: same packages, different system libraries, different behavior — or it won't even start because a `.so` library is missing. The fix is to isolate and ship the whole environment, not just the Python layer. That is **containerization**².

15.2 Containers, images, and how they differ from VMs

Two terms anchor everything, and beginners blur them:

- An **image**³ is a blueprint: a read-only, packaged filesystem containing an OS userland, the interpreter, libraries, and the app — everything needed to run. It is built once and stored.
- A **container**⁴ is a running instance of an image — a live, isolated process tree using that image's filesystem. One image, many containers (like one class, many objects — Chapter 1's distinction, applied to deployments).

The natural question: isn't this just a **virtual machine**⁵? No, and the difference is the reason Docker is practical:



A VM virtualizes hardware and runs a complete guest operating system per app — heavy and slow. A container shares the host's OS **kernel**⁶ and isolates only the userland (filesystem, processes, network) — so it is far lighter (megabytes, not gigabytes) and starts in moments. The container still gets its own copy of system libraries and interpreter, which is exactly the isolation §15.1 needed, without the weight of a whole OS. That lightness is what makes "ship the whole environment" practical for everyday use.

15.3 The Dockerfile: a recipe for an image

An image is built from a **Dockerfile**⁷ — a text recipe of instructions, each producing a **layer**⁸ (a cached filesystem diff). Common instructions: `FROM` (start from a base image), `RUN` (execute a build command, e.g. install packages), `COPY` (bring files in), `EXPOSE` (declare ports), `ENTRYPOINT` (the command to run when a container starts).

The `FROM` instruction names a **base image**⁹ — you never start from bare metal; you start from an existing image (here, an official Python image on a slim Debian) and add to it. Each instruction's layer is cached, so rebuilds skip unchanged steps — the one piece of Make-like incrementalism that is used here.

15.4 Reading `hathor-core`'s multi-stage Dockerfile

The real `Dockerfile` uses a **multi-stage build**¹⁰ — a technique that produces a small final image by doing the heavy building in a throwaway first stage and copying only the results into a clean second stage. Walk it:

Stage 0 — the builder (`Dockerfile:7`). Starts `FROM python:3.12-slim-bullseye (:7`; the version comes from `ARG PYTHON=3.12 , :3`). It then installs a lot: the runtime system libraries (`libssl1.1 graphviz librocksdb6.11 , :11`) **plus** heavy build-time tooling (`build-essential , librocksdb-dev , cargo , git , compilers — :13`) needed to compile the RocksDB bindings and other native code. It installs Poetry, sets `POETRY_VIRTUALENVS_IN_PROJECT=true (:16`, so the `.venv` lands in the project dir where it can be copied out later), copies the dependency manifests, runs `poetry install --only=main (:20`, production deps only — Chapter 13's group split paying off), copies the source, and builds + installs a wheel (`poetry build -f wheel`, then `pip-installs` it). At the end of this stage, a working environment exists — but so do all the compilers and dev headers, which you do not want in production.

Final stage — the runtime (`Dockerfile:29`). Starts `FROM` the same slim Python base again, fresh. It installs only the runtime system libraries (`libssl1.1 graphviz librocksdb6.11 , :32`) — none of the build tooling. Then the trick: it **copies the built environment from stage 0** (`:33`), leaving every compiler, header, and build artifact behind in the discarded builder. It exposes the node's ports (`EXPOSE 40403 8080 , :37` — p2p and API), and sets the entrypoint:

```
ENTRYPOINT ["python", "-m", "hathor"]           # Dockerfile:38
```

So starting a container runs the node directly. The payoff of multi-stage: the final image carries the app and its runtime libraries but not the hundreds of megabytes of build tooling — smaller to store, faster to pull, and a smaller attack surface (no compilers shipped to production).

STAGE 0 (builder) python:3.12-slim + build tools (cargo, gcc, -dev) + poetry install + build wheel (big, messy, discarded)	FINAL STAGE (runtime) python:3.12-slim (fresh) + runtime libs only COPY --from=stage-0 the venv EXPOSE 40403 8080 ENTRYPOINT python -m hathor (small, clean, shipped)
--	---

15.5 Why containerize a node, and why Docker?

Why ship a node as a container at all? A full node is infrastructure operators run on servers they don't control, against a network where behaving identically to other nodes matters. A container guarantees the operator runs the exact environment the developers tested — same interpreter, same RocksDB version, same OpenSSL — eliminating a whole class of "it behaves differently on my server" failures. It also makes deployment one command (`docker run ...`) instead of a page of system-setup instructions, and makes the node easy to run under orchestration (Kubernetes and the like) at scale.

Why Docker, and not the alternatives?

- **vs. a setup script / manual install.** A `setup.sh` that `apt install`s libraries and `pip`-installs the app drifts with the host OS and is hard to reproduce or roll back. An image is immutable and versioned — you run a specific image, and it's identical everywhere.
- **vs. a virtual machine.** A VM gives even stronger isolation but at gigabyte sizes and slow boots (§15.2). For shipping one application, that weight is unjustified; containers give enough isolation far more cheaply.
- **vs. other container runtimes (Podman, etc.).** Docker is the de facto standard with the largest tooling and registry ecosystem; the Dockerfile format it defined is understood by the alternatives anyway. Standardization is the deciding factor.

The honest trade-off: containers add a build step and a runtime engine, and share the host kernel (so they're slightly less isolated than VMs). In return you get reproducible, portable, immutable deployments that capture the entire environment — the property a node operator most needs.

15.6 Bridge — Docker across the project

Bridge — Docker in the project and the stack:

- **It is environment isolation taken to the OS level.** Where Chapter 12's `.venv` isolates Python packages, the container isolates the whole userland — same instinct, one layer down — **Chapter 12**.
- **It consumes Poetry's group split.** `poetry install --only=main` keeps dev tools out of the production image — **Chapter 13**.
- **It bundles the system libraries the node binds to.** `librocksdb` for storage (Chapter 27), `libssl` for crypto (Chapter 40) — the native deps a `.venv` can't supply.
- **make docker builds it.** The Chapter 14 target wraps the build with tag logic — **Chapter 14**.
- **The entrypoint is the node.** `python -m hathor` launches the CLI/run_node path — **Chapter 21**; the exposed ports are p2p (Chapter 34) and the API.

Recap

Concept	What it is	In the Dockerfile
image	read-only blueprint of a full environment	built by the Dockerfile
container	a running instance of an image	<code>python -m hathor</code>
vs. VM	shares host kernel; light, fast	(not a VM)
base image	the image you start FROM	<code>python:3.12-slim (:7)</code>
layer	a cached step in the build	each <code>RUN / COPY</code>
multi-stage	build heavy, ship light	<code>stage-0 (:7) → final (:29)</code>
<code>--only=main</code>	exclude dev deps from the image	<code>:20</code>
ENTRYPOINT	command run on container start	<code>:38</code>

Docker containerizes the node: it packages the interpreter, the system libraries (RocksDB, OpenSSL), the dependencies, and the app into one portable, immutable image that runs identically everywhere — solving the "works on my machine" problem one level below the virtual environment, at the level of the operating-system userland. `hathor-core`'s multi-stage Dockerfile does the heavy compiling in a throwaway builder and copies only the result into a small, clean runtime image. Containers achieve this far more cheaply than virtual machines by sharing the host kernel. This closes the

foundational-tooling cluster (Poetry, Make, Docker); the remaining Track C chapters return to the libraries the node's code calls directly, starting with how it records what it does: **structlog** and structured logging.

1. Docker is a platform for building and running containers — isolated, portable packages of an application together with its entire runtime environment (libraries, interpreter, OS userland). ↵
2. Containerization is packaging an application with everything it needs to run into a container image, so it runs consistently across different machines regardless of their host configuration. ↵
3. An image is a read-only, layered package of a filesystem (OS userland, libraries, app) — the blueprint from which containers are created. Built from a Dockerfile, stored in registries. ↵
4. A container is a running instance of an image: an isolated process (or process tree) with its own filesystem view, using the image's contents. Many containers can run from one image. ↵
5. A virtual machine emulates a complete computer, running a full guest operating system on a hypervisor. Stronger isolation than a container but far heavier, because each VM ships and boots an entire OS. ↵
6. The kernel is the core of an operating system, managing hardware, processes, and memory. Containers share the host's kernel (unlike VMs, which each run their own), which is why they are lightweight. ↵
7. A Dockerfile is a text file of instructions (`FROM` , `RUN` , `COPY` , `ENTRYPOINT` , ...) describing how to build an image. Each instruction produces a cached layer. ↵
8. A layer is the filesystem change produced by one Dockerfile instruction. Layers are cached and reused, so rebuilds skip unchanged steps; images are stacks of layers. ↵
9. A base image is the existing image a Dockerfile starts `FROM` (here an official Python-on-Debian image). You build on top of it rather than from an empty filesystem. ↵
10. A multi-stage build uses multiple `FROM` stages in one Dockerfile, doing the heavy building in early stages and copying only the needed results into a final, smaller image — leaving build tools behind. ↵